



Itinera

Planificador de viajes personalizado mediante IA generativa

Carlos Galán García · Generative AI · 2026

1. Introducción

Itinera es un planificador de viajes personalizado hecho como MVP. La idea era crear algo que una persona pudiera abrir y utilizar, y no quedarme en un chatbot que simplemente recomienda sitios. El usuario indica destino, fechas, presupuesto, intereses y alguna preferencia adicional; a partir de ahí recibe una primera propuesta organizada por días.

El límite de siete días está para que la demo sea rápida y no genere un gasto innecesario, simplemente me parecía bastante más razonable para un MVP.

2. Proceso de desarrollo del prototipo

He desarrollado la aplicación en Next.js y la he desplegado en Vercel. He preferido mantener un único proyecto, sin base de datos, agentes múltiples ni herramientas que complicaran demasiado el proyecto. La web recoge las preferencias y presenta el resultado de manera visual, con presupuesto, avisos, fuentes y opciones para hacerlo más barato, más relajado o adaptarlo a lluvia.

Para generar los itinerarios, he usado un modelo preentrenado (gpt-5.6-luna) con información recuperada fuera del modelo para obtener itinerarios realistas. He usado la API de Open-Meteo para la localización y clima; la API de OpenTripMap para lugares e imágenes; y la búsqueda web para consultar información más actual. En la práctica, esto funciona como un RAG: primero recupero contexto y después se lo paso al modelo para que redacte la propuesta. El prompt le pide separar viaje y alojamiento del gasto diario, agrupar actividades por zonas y no presentar precios u horarios como confirmados.

También he incluido algunos controles bastante sencillos: duración limitada, código de acceso opcional, claves guardadas solo en el servidor de Vercel (los tokens de las APIs), avisos de que la información puede cambiar y una respuesta de demostración si falla alguna API. La idea es reducir el trabajo inicial de organizar un viaje, no convertir la app en un sistema de reservas.

Fine-tuning con LoRA

En el enunciado del caso práctico ponía que había que hacer un ajuste del LLM para mejorar el desempeño. Como no tenía acceso al fine-tuning supervisado de OpenAI (la opción más sencilla),

he hecho un fine-tuning simple en Colab con Qwen3-0.6B y LoRA. He usado 120 ejemplos para enseñar una forma concreta de organizar los viajes: separar presupuestos, respetar el ritmo, agrupar actividades y avisar cuando un dato debe confirmarse.

El resultado ha sido útil en cuanto a organización del viaje; después del ajuste, el modelo pasa de cumplir 4 de 7 reglas de estructura a cumplir las 7. Sin embargo, también se vuelve más rígido: el modelo base suena más natural, aunque inventa sitios y detalles con bastante seguridad. La mejora real es que sigue más consistentemente las reglas que quería para Itinera.

No he implementado el modelo LoRA en la aplicación final. Integrar este modelo habría requerido alojar tanto Qwen como el adaptador en un servicio de inferencia (lo he intentado en HF, pero necesitaba cuenta de pago), y creo que realmente añade demasiado lío para un MVP. Además, la aplicación necesita información actualizada, y aquí GPT junto a la búsqueda web sigue siendo bastante más útil.

3. Entregables

Aplicación funcional:

<https://itinera-omega.vercel.app/>

Repositorio con el código, README, documentación y notebook ejecutado:

<https://github.com/carlosgalan01/itinera>

4. Aplicación práctica

La aplicación sirve para organizar una primera propuesta de viaje. En vez de abrir muchas pestañas solo para tener una idea de cómo repartir un viaje, el usuario recibe una ruta organizada, una estimación separada del presupuesto y enlaces que puede revisar antes de reservar.

En mi opinión, la parte más interesante no es que el modelo “organice un viaje” por sí solo. Es que usamos información disponible en la web con un modelo de generación de texto que funciona como si fuera un agente de viajes. Este mismo patrón se podría trasladar a asistentes internos, herramientas de atención al cliente o sistemas que necesiten explicar de dónde ha salido una respuesta.

5. Bibliografía

Hugging Face. (2026). PEFT: LoRA. https://huggingface.co/docs/peft/package_reference/lora

Hugging Face. (2026). TRL: SFT Trainer. https://huggingface.co/docs/trl/sft_trainer

OpenAI. (2026). Web search tool. <https://developers.openai.com/api/docs/guides/tools-web-search>

Open-Meteo. (s. f.). Weather Forecast API. <https://open-meteo.com/en/docs>

OpenTripMap. (s. f.). API documentation. <https://dev.opentripmap.org/>